

Modular Programming



Handling a complex code is challenging. To simplify this, we need a technique that breaks large code into smaller sections based on their functionality.



Every day of our lives is made up of multiple activities. We play sports, watch movies, listen to music, study, cook a meal, and more. We do each activity step-by-step.

For example, while cooking we first complete the prep, which involves cutting vegetables, kneading dough, and steaming rice. After this comes the actual cooking. Each of these individual tasks are also divided into smaller tasks.

Programming works the same way. When we proceed with a large program, it becomes complex to understand and execute. It is important that the code we write is neat and easy to write and read. We use modular programming for this. Modular programs are simpler to understand and maintain. This also reduces errors and helps easily find the bugs in the code.

9. Modular Programming

Modular Programming

Modular programming organises a code into smaller parts. Each part contains the code required to run one specific part of the final program.

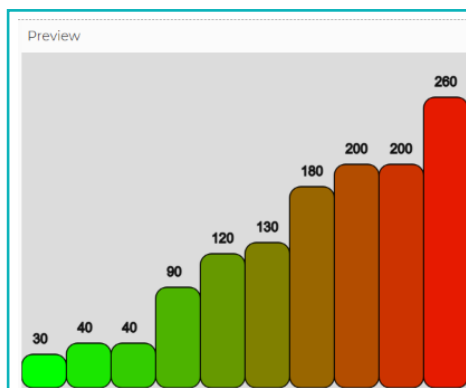


Recall:

In the last lesson we implemented a bubble sort algorithm. We wrote a code to create columns of random heights. When the mouse button was clicked, the columns arranged themselves by ascending order of height.

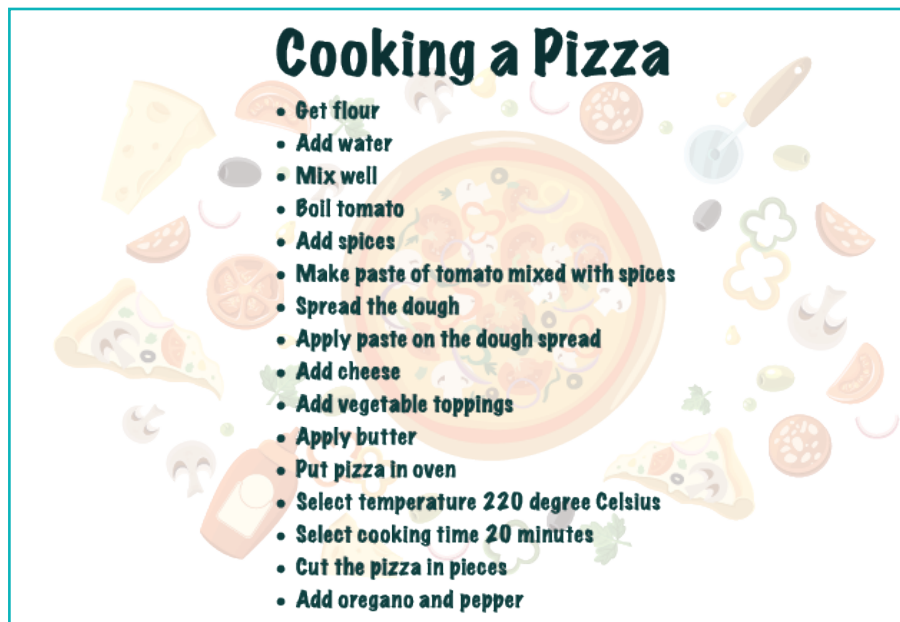


In this lesson, we will use modular programming to make the same code easy to write and read. We'll add some visual effects to make it look pleasing. We can fill columns with colours of a changing gradient as per their order.

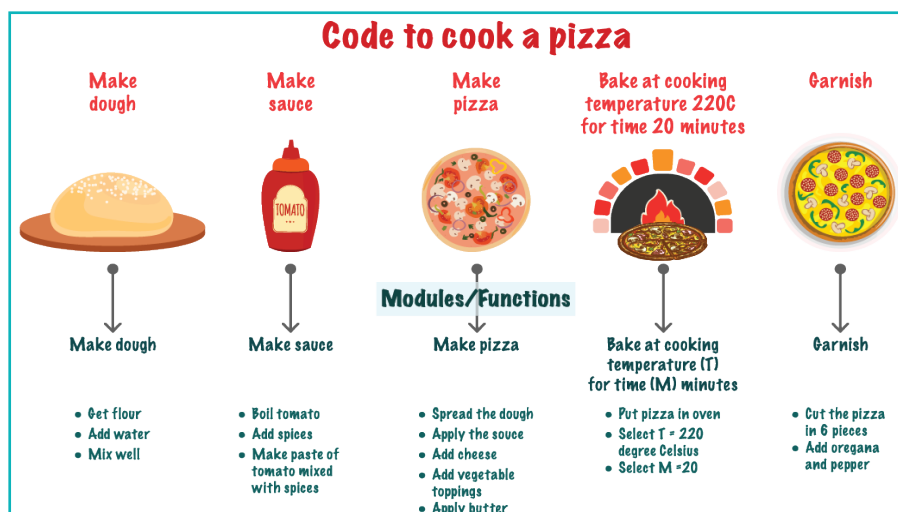


To understand modular programming in more detail, let's consider the following example:

Imagine a restaurant where robots cook meals. If they want to cook a pizza, what steps would they follow?



This procedure looks long and tedious to follow. Instead, if we follow the concept of modular programming, we can create a group of actions to perform specific tasks like making a dough, making sauce, baking etc.



This divides code into simpler steps. It looks neat, clean and easy to understand. If the robot chefs need to bake bread for a sandwich, they can use premade modules to make a dough and bake it. The code does not have to be rewritten.

To relate modular programming to our code, remove all comments in the code we created last time.

```
1 var columns = [];  
2 var count;  
3  
4 function setup() {  
5   createCanvas(400, 300);  
6   count = 5;  
7   for(let i=0; i<count; i=i+1){  
8     columns[i] = random(height);  
9   }  
10 }  
11  
12 function draw() {  
13   console.log(columns.length);  
14   background(220);  
15   fill(255);  
16   for(let i=0; i < columns.length; i=i+1){  
17     rect(i*width/count, height-columns[i],width/count,columns[i],10);  
18   }  
19 }  
20  
21 function mousePressed(){  
22   for(let i = count; i > 0; i=i-1){  
23     for(let j = 0; j < i; j=j+1) {  
24       if(columns[j] > columns[j+1]) {  
25         //Swap code  
26         let temp = columns[j];  
27         columns[j]=columns[j+1];  
28         columns[j+1]=temp;  
29       }  
30     }  
31   }  
32 }
```

We can divide this code into different sections:

1. **Create columns array-** Use for loop to add random values in columns array
2. **Draw columns-** Add colour and draw rectangles using for loop
3. **Sort algorithm-** Add a nested for loop and condition for the sorting algorithm
4. **Swap algorithm-** Add a swap algorithm code

```
1 var columns = [];  
2 var count;  
3  
4 function setup() {  
5   createCanvas(400, 300);  
6   count = 5;  
7  
8   for(let i=0; i<count; i=i+1){  
9     columns[i] = random(height);  
10 }  
11 }  
12  
13  
14 function draw() {  
15   background(220);  
16  
17   fill(255);  
18   for(let i=0; i < columns.length; i=i+1){  
19     rect(i*width/count, height-columns[i],width/count,columns[i],10);  
20 }  
21 }  
22  
23  
24 function mousePressed(){  
25  
26   for(let i = count; i > 0; i=i-1){  
27     for(let j = 0; j < i; j=j+1) {  
28       if(columns[j] > columns[j+1]) {  
29  
30         let temp = columns[j];  
31         columns[j]=columns[j+1];  
32         columns[j+1]=temp;  
33       }  
34     }  
35 }  
36 }  
37  
38 }
```

Exercise

Tick mark ☒ the correct answer

1 A program can be divided into different parts based on the tasks performed. Such an approach in coding is called _____ .

- | | | | |
|--------------------------------|--------------------------|----------------------|--------------------------|
| a. Modular Programming | ☐ | b. Scripting | ☐ |
| c. Object-Oriented Programming | ☐ | d. None of the above | ☐ |

2 What are the advantages of modular programming?

- | | | | |
|-----------------------|--------------------------|-----------------------|--------------------------|
| a. Easy To Understand | ☐ | b. Simple to maintain | ☐ |
| c. No Repetition | ☐ | d. All of the above | ☐ |

We can use functions to create separate modules for these four sections. We have learnt about default functions like `setup()`, `draw()`, `mousePressed()` etc. All functions have a predefined task.

We can create functions on our own, called user defined functions. Functions can be created for all four modules of code we discussed. We can also use the functions in any other code whenever required like swap function.

Function: A function is a set of code that performs a specific task. It usually has an input and an output.

For example: In a coffee vending machine, sugar, milk and coffee is the input. As an output, we get a latte or a cappuccino.



Similarly, in programming, functions might take a number, character or variable value as the input and give the required output.

For example: The following function takes two numbers as the input and performs an arithmetic operation such as multiplication between them to give an output.

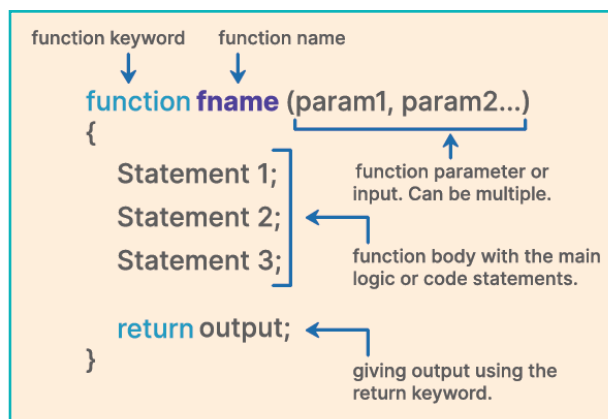
```
function multiply2num(a,b){  
    let answer = a * b;  
    return answer;  
}
```

We can execute the code and console will show the output as the multiplication of the two numbers given as input to the function.

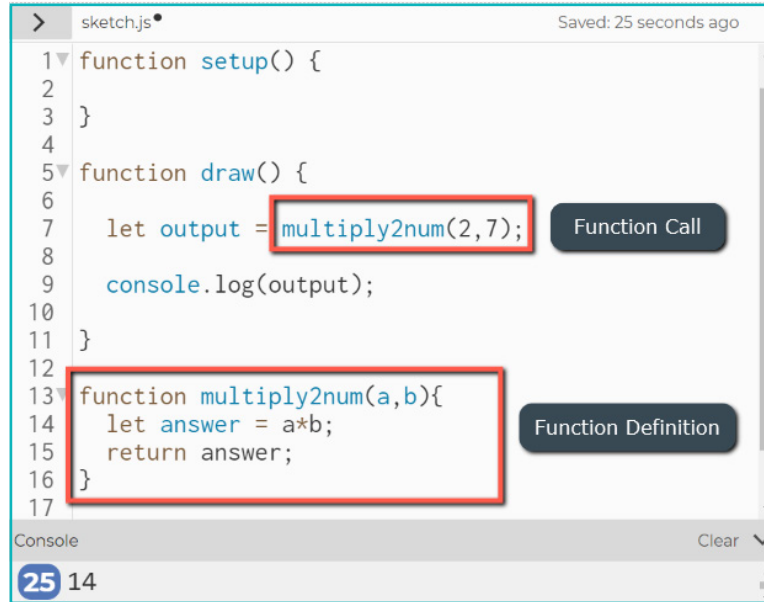
There are two parts of a function—function definition and function call.

Function definition: Here, we add code about the specific task that the function will perform. Every function definition starts with the word ‘function’, followed by the name of the function, a pair of round brackets and a pair of curly brackets.

```
function functionName () {  
    //We write code here  
}
```



To use the function, we call the function in `draw()` by writing the function name and adding inputs if any. This is referred to as calling a function or a function call.



```
> sketch.js • Saved: 25 seconds ago
1 function setup() {
2
3 }
4
5 function draw() {
6   let output = multiply2num(2,7);
7   console.log(output);
8 }
9
10
11
12
13 function multiply2num(a,b){
14   let answer = a*b;
15   return answer;
16 }
17
```

Function Call

Function Definition

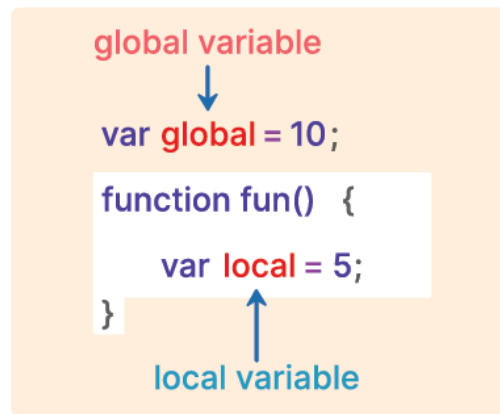
Console 25 14 Clear

Note

It is not necessary for a function to take an input and give an output. There can be a function without any input or a function that doesn't return any specific output.

Global and Local variables

We know that a function definition has variables as input parameters, if an input is provided to a function. The scope of such variables is limited only to that function. These variables don't exist beyond the scope of the function. Variables with a scope limited to the snippet of function code are called **local variables**. Usually, local variables are temporary variables required only in the scope of a specific function of the code structure.



Variables declared outside of functions like `setup()` and `draw()` which can be used anywhere in the program are called **global variables**. They are usually at the beginning of the code.

Exercise

State whether True or False

3 A code can be divided into subsections called functions based on their functionalities.

 _____

4 Each function starts with the keyword 'void'.

 _____

5 A function can have input parameters as the input and give an output with a return keyword.

 _____

6 Variables declared outside a function at the beginning of the code, whose scope extends all over the code, are called local variables.

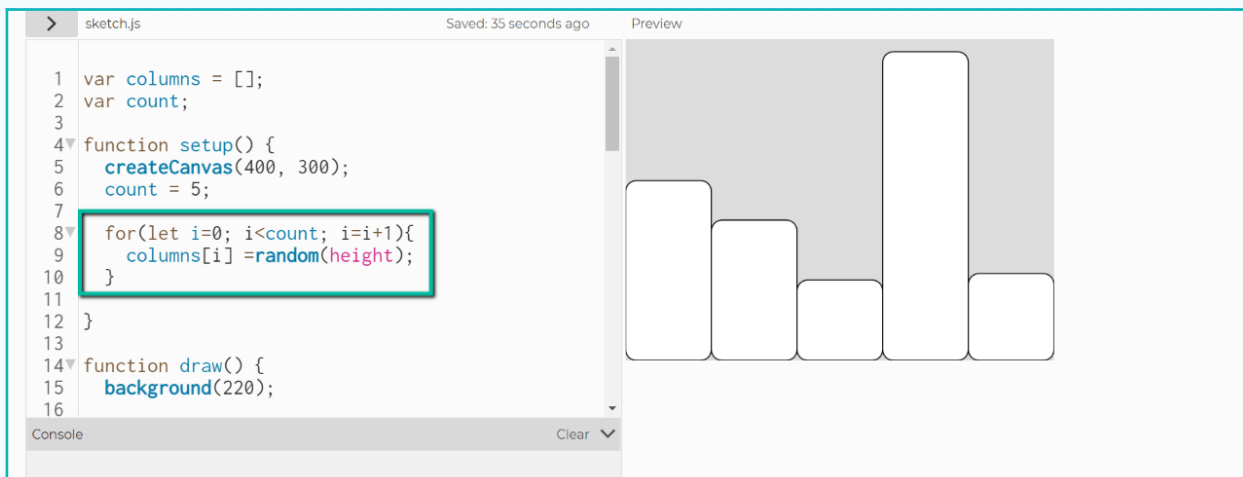
 _____

Activity

■ Create functions for each section of code and fill the columns with a gradient according to their order.

1 Identify code for each function

- Open the code for sorting from the earlier lesson in p5.js.
- Check if code is working seamlessly and remove all the comments.
- Identify the first section of code which adds random height values in the columns array to create a function.



2 Create a new function to add random height values in the columns array

- Create a new function called `createColumnsArray(arrayLength)` using keyword “function” and an input parameter “arrayLength”. Place it after `mousePressed()` function code block.

Activity

- b. We can provide an input parameter as 'count' variable at the time of the function call to use in for loop to decide how many array elements or columns we want. Value provided as input i.e in count variable during function call will be saved inside the function local variable 'arrayLength'.

```
24 for(let i = count; i > 0; i=i-1){
25   for(let j = 0; j < i; j=j+1) {
26     if(columns[j] > columns[j+1]) {
27
28       let temp = columns[j];
29       columns[j]=columns[j+1];
30       columns[j+1]=temp;
31     }
32   }
33 }
34 }
35 }
36 }
37 }
38 function createColumnsArray(arrayLength){
39
40
41 }
42 }
```

- c. Cut the code for adding random height values in the columns array and paste in the function.
- d. Change the condition "i < count" to "i < arrayLength" as the input given to the function in the function call will be stored in arrayLength variable.

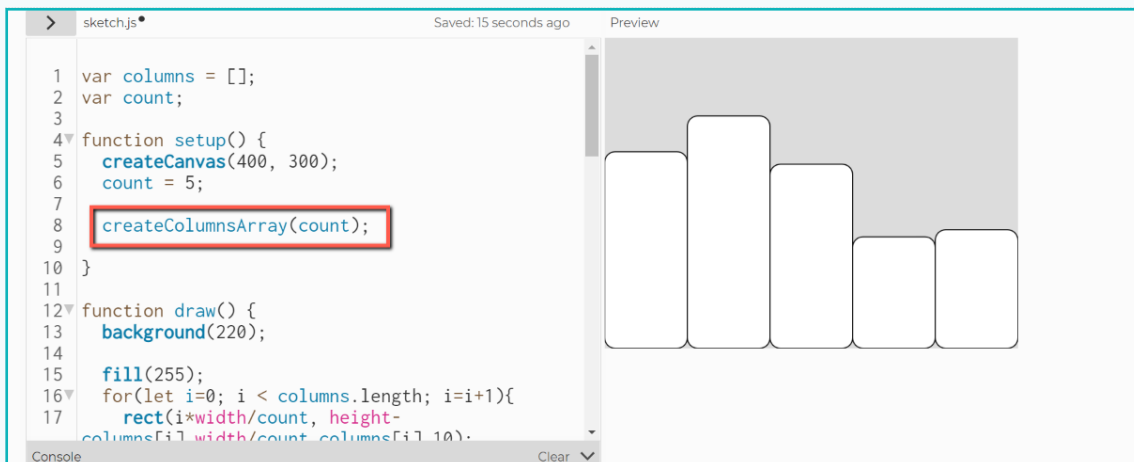
```
21
22
23 let temp = columns[j];
24 columns[j]=columns[j+1];
25 columns[j+1]=temp;
26
27 }
28 }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 function createColumnsArray(arrayLength){
39
40   for(let i=0; i<arrayLength; i=i+1){
41     columns[i] = random(height);
42   }
43 }
44 }
45 }
```

We can't preview the columns as we have just defined the function and not called it yet.

Activity

3 Call the function

- Call the function in `setup()` where the for loop code to create a random column was written.
- Use `count` variable as an input in the function call.

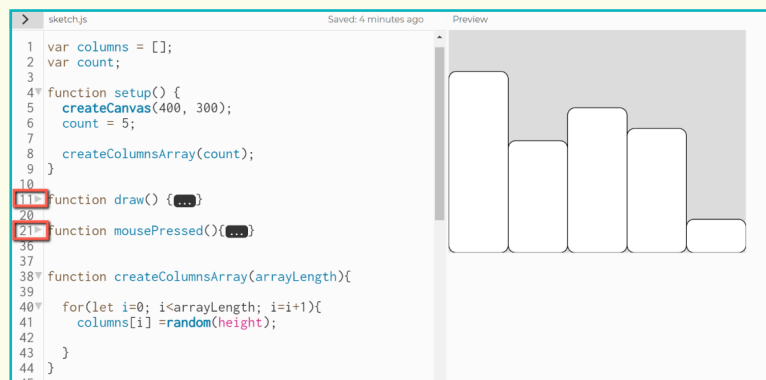


```
1 var columns = [];  
2 var count;  
3  
4 function setup() {  
5   createCanvas(400, 300);  
6   count = 5;  
7   createColumnsArray(count);  
8 }  
9  
10  
11  
12 function draw() {  
13   background(220);  
14  
15   fill(255);  
16   for(let i=0; i < columns.length; i=i+1){  
17     rect(i*width/count, height-  
18       columns[i].width/count, columns[i].width, 10);  
19   }  
20 }  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45
```

The columns are now displayed in the output preview.

Note

Click on the small arrows near line numbers 11 and 21 to minimize snippets of the code if those part does not need any attention.

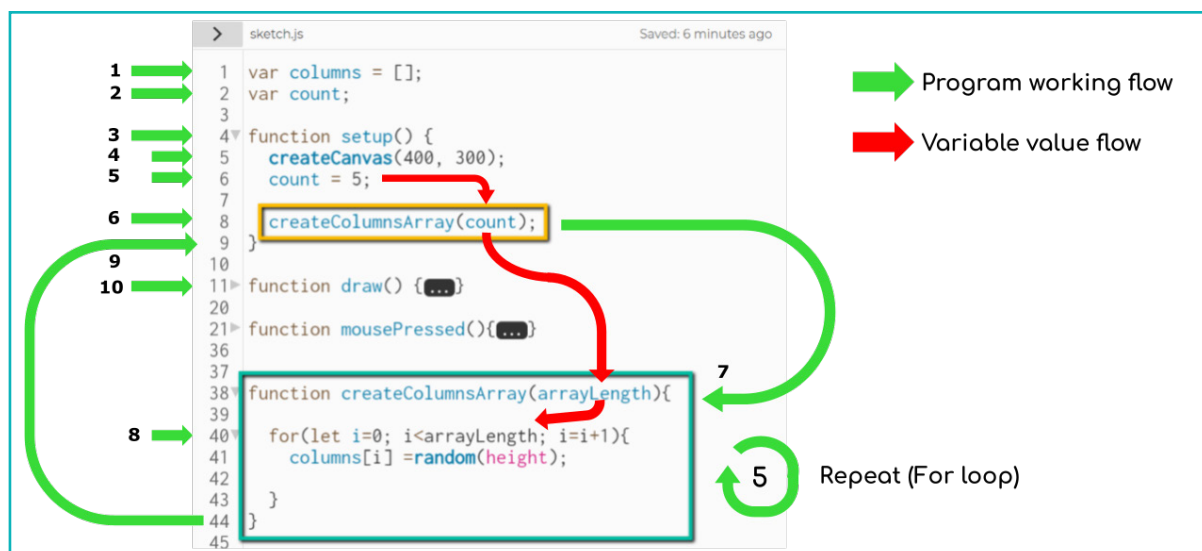


```
1 var columns = [];  
2 var count;  
3  
4 function setup() {  
5   createCanvas(400, 300);  
6   count = 5;  
7   createColumnsArray(count);  
8 }  
9  
10  
11  
12 function draw() {  
13   background(220);  
14  
15   fill(255);  
16   for(let i=0; i < columns.length; i=i+1){  
17     rect(i*width/count, height-  
18       columns[i].width/count, columns[i].width, 10);  
19   }  
20 }  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45
```

Bytes

Program Control Flow: In computer programming, it is the order in which code instructions or statements are executed when a program is running.

How do functions work? Usually, the program executes code line by line. Here, the code is running as per the function calling. Look at the image below. Numbers show the sequence of the code execution, which are different from the code line number.



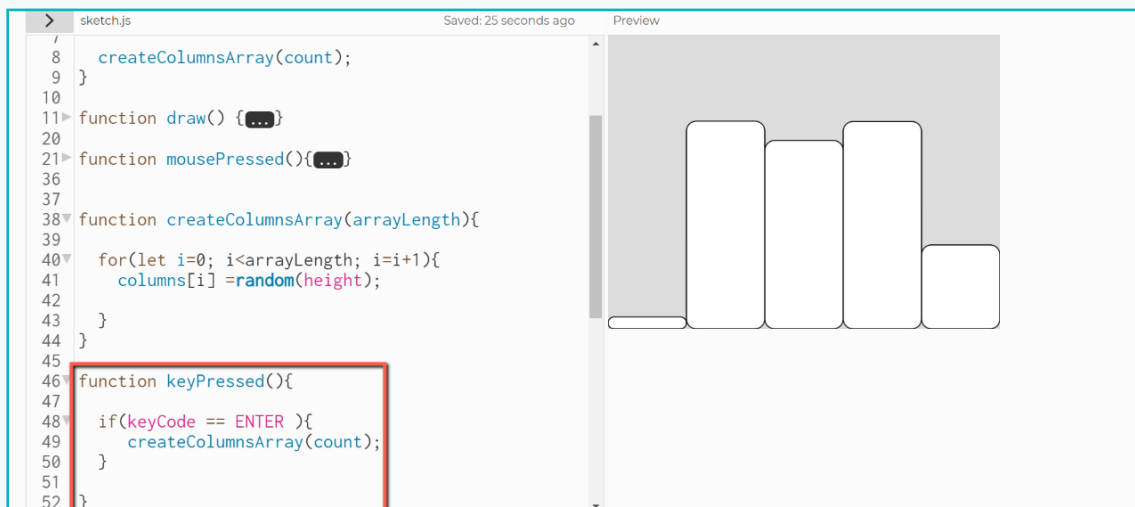
- When the 6th step in the code on line 8 for function call is executed, the program execution flow goes to the function definition at line 38.
- The function has a for loop which runs 5 times as the value in count variable is taken as input.
- The value comes to 'arrayLength' local variable.
- Once all the code in `createColumnsArray()` function is executed, control flow goes back to the next line of the function call.

We can use this function to create a functionality so that if we press 'enter' on the keyboard, a new set of columns with random heights will appear in a random order. This way, we do not need to rerun the code to check for ascending code.

Activity

4 Create a function to add values in columns array

- Add default function keyPressed() to detect when a key is pressed.
- Add if condition to check if the key pressed is 'Enter'. Do this by checking if default variable keyCode has value "ENTER".
- Call function createColumnsArray(count) with count as the input parameter inside if condition block. This will add new height values in the array.



```
1 > sketch.js Saved: 25 seconds ago Preview
2 /
3   createColumnsArray(count);
4 }
5
6 10
7 11 function draw() {...}
8 20
9 21 function mousePressed(){...}
10 36
11 37
12 38 function createColumnsArray(arrayLength){
13 39
14 40   for(let i=0; i<arrayLength; i=i+1){
15 41     columns[i] =random(height);
16 42   }
17 43 }
18 44 }
19 45
20 46 function keyPressed(){
21 47
22 48   if(keyCode == ENTER ){
23 49     createColumnsArray(count);
24 50   }
25 51 }
26 52 }
```

Note

The default variable keycode stores the code of the key pressed on the keyboard. It also detects special key values such as 'BACKSPACE', 'DELETE' and 'ENTER'.

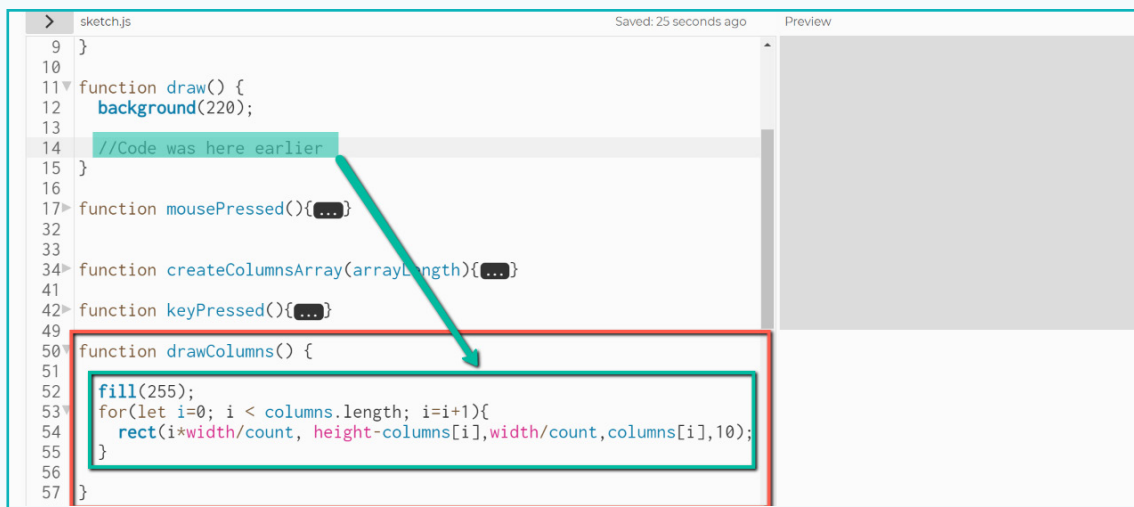
Now, we can just rearrange the unsorted columns by pressing Enter. We do not need to run the code again. Here, we did not have to write the complete code to add new random values to columns array. Instead, we just called the function we created earlier. This saved us the effort of writing lines of repetitive code.

Let's create functions for the other modules we discussed.

Activity

5 Create a new function to render the columns

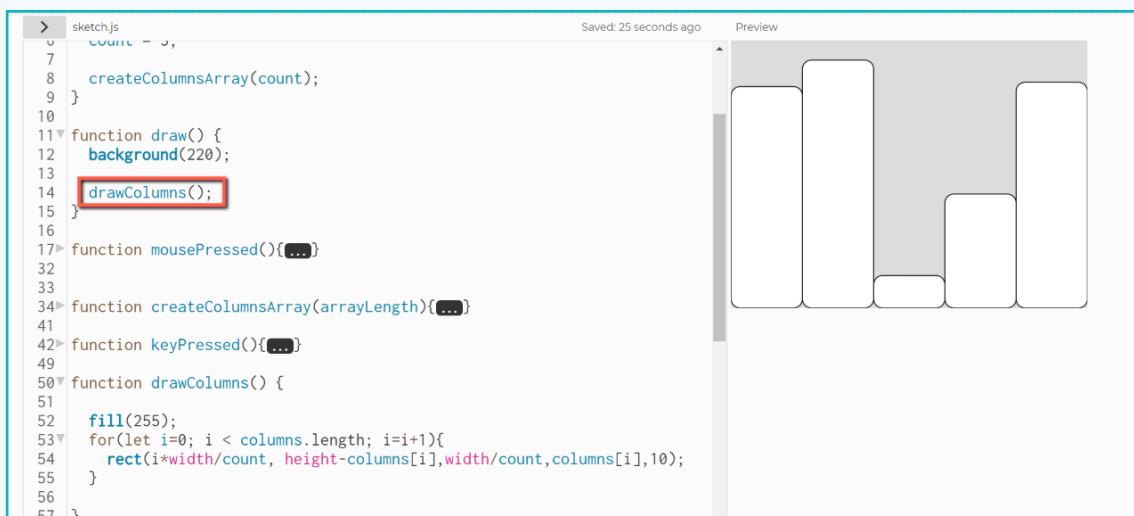
- Create a new function called `drawColumns()` to draw columns using height values in the array.
- Cut the code that draws columns and add it in new `drawColumns()` function.



```
9 }
10
11 function draw() {
12   background(220);
13
14   //Code was here earlier
15 }
16
17 function mousePressed(){...}
32
33
34 function createColumnsArray(arrayLength){...}
41
42 function keyPressed(){...}
49
50 function drawColumns() {
51   fill(255);
52   for(let i=0; i < columns.length; i=i+1){
53     rect(i*width/count, height-columns[i],width/count,columns[i],10);
54   }
55 }
56
57 }
```

We can't preview the columns as we have just defined the function and not called it yet.

- Call the function in `draw()` function where the original code was saved.
- Note that the function does not have any input parameters.



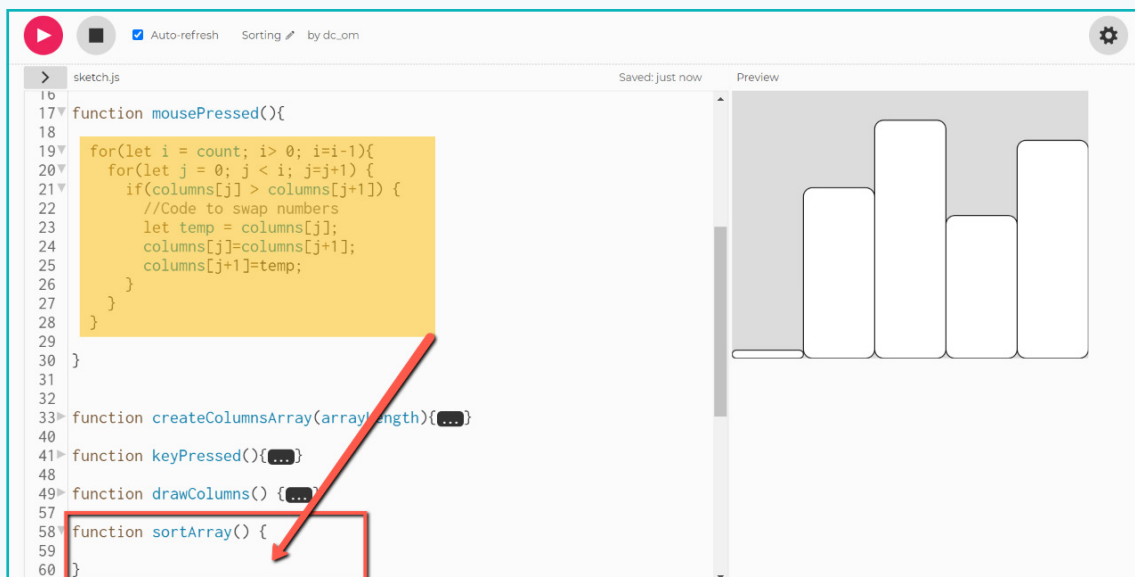
```
6 count = 5,
7
8   createColumnsArray(count);
9 }
10
11 function draw() {
12   background(220);
13   drawColumns();
14 }
15
16
17 function mousePressed(){...}
32
33
34 function createColumnsArray(arrayLength){...}
41
42 function keyPressed(){...}
49
50 function drawColumns() {
51   fill(255);
52   for(let i=0; i < columns.length; i=i+1){
53     rect(i*width/count, height-columns[i],width/count,columns[i],10);
54   }
55 }
56
57 }
```

Activity

6 Create a function to sort array using bubble sort algorithm

Create another function called `sortArray()` for the third module

- Create a new function `sortArray()` and
- Cut and paste code written for sorting the array from `mousePressed()` function and paste in the `sortArray()` function.



Call `sortArray()` function inside `mousePressed()`. When the mouse key is pressed, it will call the `sortArray()` function.



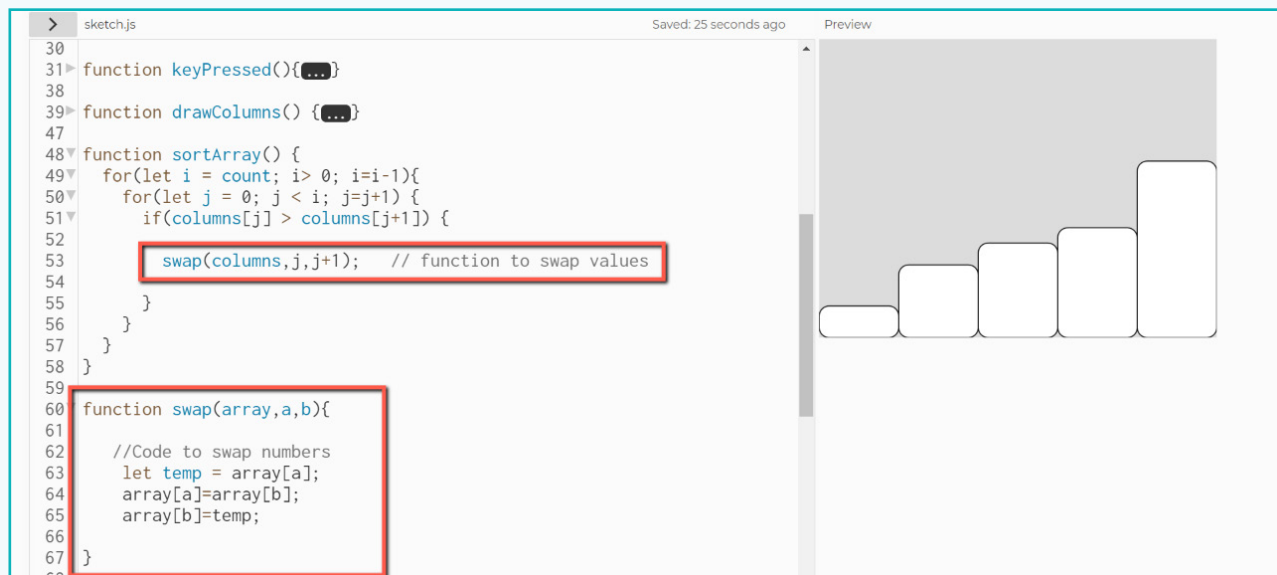
Activity

7 Create a function to swap values as the last module

Try to create a function for swap algorithm to swap the values in two variables.

Hint

We need to provide array and array indices of the elements to be swapped as function input parameters. If you get stuck, follow the image.



```
30
31 function keyPressed(){...}
38
39 function drawColumns() {...}
47
48 function sortArray() {
49   for(let i = count; i > 0; i=i-1){
50     for(let j = 0; j < i; j=j+1) {
51       if(columns[j] > columns[j+1]) {
52         swap(columns,j,j+1); // function to swap values
53       }
54     }
55   }
56 }
57 }
58 }
59
60 function swap(array,a,b){
61
62   //Code to swap numbers
63   let temp = array[a];
64   array[a]=array[b];
65   array[b]=temp;
66 }
67
68
```

The screenshot shows a code editor with a file named 'sketch.js'. The code defines several functions: `keyPressed()`, `drawColumns()`, and `sortArray()`. Inside `sortArray()`, there is a call to `swap(columns,j,j+1);` which is highlighted with a red box. Below it, the `swap(array,a,b)` function is defined, also highlighted with a red box. The `swap` function uses a temporary variable `temp` to swap the values at indices `a` and `b` in the `array`. To the right of the code editor is a 'Preview' window showing a bar chart with five bars of increasing height.

Note

Function parameters given as input in function call are stored in local variables like, 'a' and 'b'.

With this we've finished creating a modular code. We added different functions for each category of tasks, which can be called in different parts of the code. Our final code is easy to read, understand and follow.

Exercise

Answer in one line based on the image provided

```
1 function setup() {  
2   createCanvas(400, 400);  
3 }  
4  
5 function draw() {  
6   background(220);  
7   var x = addition(5,9);  
8   console.log(x);  
9 }  
10  
11 function addition(a,b){  
12   return (a+b);  
13 }  
14  
15  
16  
17  
18
```

a

b

c

d

e

7 What is the meaning of 'program control flow'?



8 In what sequence will the above code be executed? Answer format: (b-a-c-e-d)



9 Is there any error in the code? If no, what will be the output displayed in the console?



Bytes

While developing an application or game, many programmers simultaneously work on thousands of lines of code. To avoid ambiguity and error, the code needs to be easy to read and understand. Using proper indentation and comments to explain the code is a good practice.



Activity

8 Create arrays to save colour values for columns

Hint:

Create 3 arrays `r[]`, `g[]`, and `b[]` to store red, green and blue colour values. Each array element in `r,g,b` array is a red, green and blue colour value out of 255 for each column.

```
sketch.js Saved: 15 seconds ago
1 var columns = [];
2 var count;
3
4 var r = [];
5 var g = [];
6 var b = [];
7
8 function setup() {
9   createCanvas(400, 300);
10  count = 5;
11
12  createColumnsArray(count);
13 }
14
15 function draw() {
16   background(220);
17   drawColumns();
18 }
19
20
21 function mousePressed() {
22   // ...
23 }
24
25
26
27 function createColumnsArray(arrayLength) {
28   // ...
29 }
Console Clear
```

Activity

Add initial values of 255 in the `r[]`, `g[]`, `b[]` arrays in `createColumnsArray()` function inside for loop with 'i' as array index. This will set the initial colour of all columns to white.

```
12 createColumnsArray(count);
13 }
14
15 function draw() {
16   background(220);
17   drawColumns();
18 }
19
20
21 function mousePressed(){...}
26
27 function createColumnsArray(arrayLength){
28   for(let i=0; i<arrayLength; i=i+1){
29     columns[i] = random(height);
30
31     r[i] = 255;
32     g[i] = 255;
33     b[i] = 255;
34   }
35 }
36
37 }
38
39 function keyPressed(){...}
```

9 Add colour to the columns using values in colour Array

Instead of `fill(255)` command in `drawColumns()` function; add `fill(r[i],g[i],b[i])`, command inside for loop before `rect()` command to draw function. This is to add colours from the `r,g,b` arrays we created. As all values in `r,g,b` arrays are 255, all columns will be white.

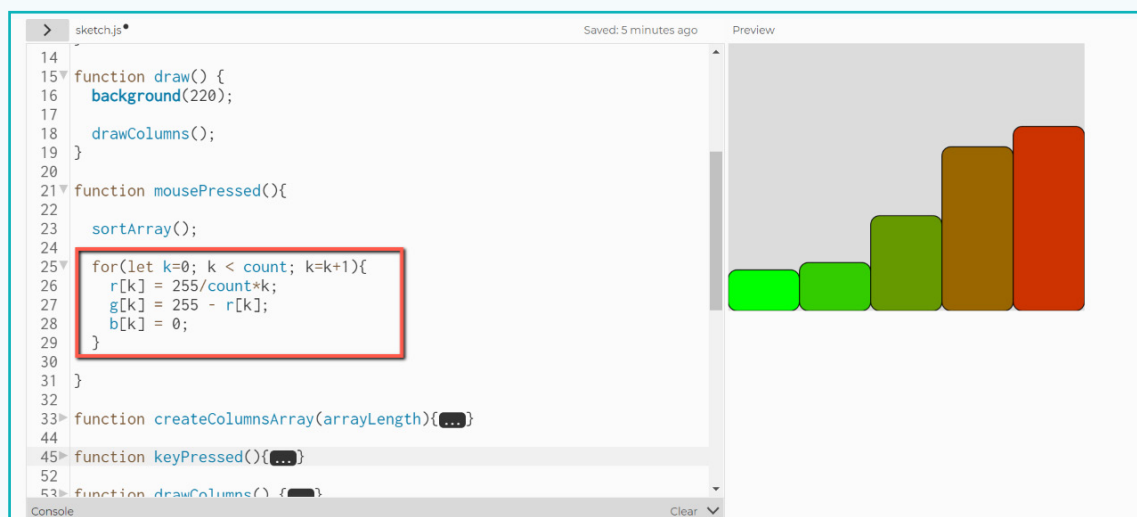
```
39 g[] = 255;
40 b[] = 255;
41 }
42 }
43 }
44
45 function keyPressed(){...}
52
53
54 function drawColumns() {
55   //fill(255);
56   for(let i=0; i < columns.length; i=i+1){
57     fill(r[i],g[i],b[i])
58     rect(i*width/count, height-columns[i],width/count,columns[i],10);
59   }
60 }
61
62 }
63
64 }
65
66 function sortArray() {...}
77
78 function swap(array,a,b){...}
```

Activity

10 Update the values in colour arrays to have gradient colors for sorted columns

When mouse button is clicked to sort the array elements, the colours should be a gradient from green to red indicating increasing height. Initially values in `r[]` array should start from 0 and reach up to 255 as 'k' increases from 0 to array length i.e. 5. Whereas, values in `g[]` array should start from 255 and reach up to 0 as 'k' increases from 0 to 5. Values in `b[]` array should become 0.

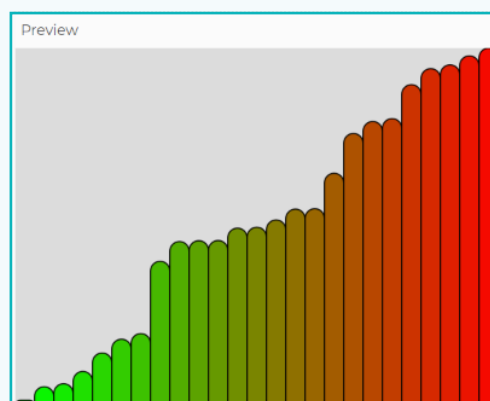
- Add code to add a colour gradient to columns in `mousePressed()` function.



```
14 > sketch.js
15 function draw() {
16   background(220);
17   drawColumns();
18 }
19
20
21 function mousePressed(){
22   sortArray();
23
24   for(let k=0; k < count; k=k+1){
25     r[k] = 255/count*k;
26     g[k] = 255 - r[k];
27     b[k] = 0;
28   }
29 }
30
31 }
32
33 function createColumnsArray(arrayLength){...}
34
35 function keyPressed(){...}
36
37 function drawColumns(){...}
```

The screenshot shows a code editor with a JavaScript file named `sketch.js`. The code defines a `draw()` function that calls `background(220)` and `drawColumns()`. A `mousePressed()` function is also defined, which calls `sortArray()` and then loops through the `count` array, updating the `r`, `g`, and `b` color arrays to create a gradient from green to red. The `r` array starts at 0 and increases to 255, while the `g` array starts at 255 and decreases to 0. The `b` array is set to 0. The `drawColumns()` function is also shown, which uses these arrays to draw the columns. The preview window on the right shows a bar chart with five columns of increasing height, colored in a gradient from green to red.

When the columns are sorted, they are coloured in a gradient. Even if we increase the number of columns by changing the value in `count` variable, the code will work the same way.



9. Modular Programming

In this lesson, we learnt that modular programming is more than just a way to write code. It is also an efficient method to use when multiple coders are working on the same application. We also learnt how a user can create customized functions and use them to get the desired output.

Functions ensure that the code is neat and clean. They can also be used in a code repeatedly to negate the possibility of writing more code.



Tech Flash

- **Modular programming** : a process of breaking a code into independent modules which have specific tasks.
- **Function** : a block of code which is allocated a specific task and can be used in any code without repeating all the steps in the code block.
- **Global variable** : a variable whose scope extends to the entire code.
- **Local variable** : a variable with scope limited to a specific code block or function.
- **Program control flow** : the order in which code statements are executed when the code is run.